

Functional Plugin Extraction and Soft Superposition Injection via Parameter Deltas from Homologous Language Models

Shizhe Jin

Independent Researcher

Abstract: Large language models accumulate different capability increments during pre-training, supervised fine-tuning (SFT), and reinforcement learning (RL) stages, but these increments are typically entangled and non-reusable. This paper proposes a simple method to extract "functional plugins" by computing parameter-wise deltas between homologous model checkpoints, and injects them into the base model via soft superposition (scaling coefficient α). On the poetry generation task of MiniCPM-1B, we find that: (1) directly extracting RL-Base delta outperforms chained subtraction; (2) MLP layers are safe carriers for injection, while Attention layers trigger repetition collapse; (3) the optimal configuration is MLP-only injection with $\alpha=0.3$. Our method provides a feasible path toward modular AI systems.

Keywords: Parameter Deltas; Soft Superposition; Plugin Injection; Instruction Following

1 Introduction

Large language models typically undergo multiple training stages: pre-training (Base) learns broad linguistic knowledge, supervised fine-tuning (SFT) equips them with instruction-following capabilities, and reinforcement learning (RL) further refines their output strategies. However, the capability increments acquired at each stage are coupled within the model, making them difficult to separate and reuse. If the capabilities of a particular stage (e.g., instruction following, format control) could be extracted as independent plugins and flexibly injected into other base models, it would greatly enhance the flexibility and efficiency of model deployment.

Existing research has attempted to achieve capability transfer through parameter deltas (e.g., Task Vectors), but primarily focuses on horizontal transfer across different downstream tasks. This paper focuses on the vertical evolution of the same model along its training timeline, and discovers that directly extracting stage-wise deltas faces distribution shift issues. To address this, we propose a soft superposition injection protocol and validate its effectiveness on a poetry generation task.

The main contributions of this paper are:

- (1) A method to extract functional plugins via parameter deltas from homologous models without additional training;
- (2) The finding that directly extracting the final-stage delta (RL-Base) is superior to chained subtraction, which accumulates floating-point errors;

- (3) A soft superposition injection protocol and validation that MLP layers are safe carriers for injection;
- (4) Best practice parameters (MLP-only, $\alpha=0.3$), providing an engineering pathway for modular AI.

2 Related Work

Parameter-efficient fine-tuning methods (e.g., LoRA, Adapter) adjust model behavior by training small-scale incremental matrices, but require additional computational resources and data. Task Vector methods extract task increments by subtracting the pre-trained model from fine-tuned models and can add or subtract them to other models. Model merging methods (e.g., TIES-Merging) attempt to fuse the weights of multiple fine-tuned models to achieve better generalization.

The differences between our method and the above approaches are: LoRA/Adapter derive increments from training and require training, while ours is training-free; Task Vectors derive increments from cross-task model subtraction, which does not require training but requires fine-tuning two models, whereas ours focuses on the time-stages of the same model; TIES-Merging achieves fusion by averaging multiple models, while ours pursues pluggability rather than fusion. Our method is the first to focus on vertical temporal evolution and introduces a soft superposition protocol.

3 Method

3.1 Delta Extraction

Given two model weights W_{high} and W_{low} of the same architecture (representing later and earlier training stage checkpoints, respectively), we compute their layer-wise, parameter-wise difference:

$$\Delta = W_{\text{high}} - W_{\text{low}}$$

This delta represents the capability increment acquired during the training stage. We refer to it as a "functional plugin."

3.2 Soft Superposition Injection

When injecting the extracted delta into the base model W_{base} , we adopt a soft superposition protocol:

$$W_{\text{new}} = W_{\text{base}} + \alpha \times \Delta$$

where $\alpha \in (0, 1]$ is the scaling coefficient. We found that $\alpha=1$ (hard superposition) leads to distribution shift and generation collapse, while reducing α effectively mitigates this issue.

3.3 Layer Selection Strategy

We further discovered that different network layers have different tolerance to injection. Based on experimental validation, we adopt the following strategy: inject only MLP layers (including gate_proj, up_proj, down_proj), and do not inject Attention layers (including q_proj, k_proj, v_proj, o_proj). MLP layers primarily store associative knowledge, and their weight distributions have high redundancy and robustness; Attention layers are highly sensitive to numerical magnitude, and even small shifts can cause attention collapse.

4 Experiments

4.1 Experimental Setup

We use the MiniCPM-1B model family, including Base (pre-training), SFT (supervised fine-tuning), and RL (reinforcement learning) versions. The task is five-character quatrain generation, with the prompt: "Please write a five-character quatrain about winter, consisting of four lines, each with five characters, rhymed." Decoding uses beam search (num_beams=4) without random sampling.

4.2 Baselines

Base (no plugin) outputs free verse, unformatted, indicating knowledge without instruction following. Original SFT outputs complete quatrains with annotations, serving as the standard reference. Original RL outputs complete quatrains with annotations and greater rigor, serving as a reference baseline.

4.3 Scaling Coefficient Sweep

We fix the MLP-only plugin (RL-Base delta, MLP layers only) and test output quality under different α values. At $\alpha=0.001$, output is a mix of six- and seven-character lines, with hallucinated dialogue, indicating failure (insufficient injection). At $\alpha=0.3$, output is "Winter snow flutters across the sky, Snowflakes drift gently down, Cold wind howls piercingly, Snowflakes dance in the air." indicating success. At $\alpha=0.5$, output is five-character but with repetition of "snowflakes dance in the air," indicating partial success. At $\alpha=0.7$, repetition intensifies, indicating failure. At $\alpha=1.0$, infinite loop occurs, indicating failure. Results show that $\alpha=0.3$ is the optimal choice, enabling stable, correctly formatted generation.

4.4 Comparison of Different Plugin Types

We further compared plugins from different extraction methods and layers. MLP-only (RL-Base) using single subtraction yields stable output, optimal. Attention-only (RL-Base) using single subtraction yields immediate repetition loop. Full plugin (RL-Base) using single subtraction yields repetition collapse. Chained plugin (RL-SFT + SFT-Base) using two-step subtraction yields semantic collapse and irrelevant content. The key finding is: directly extracting RL-Base is superior to chained subtraction. The latter, while mathematically equivalent, accumulates floating-point errors across two operations, which is sufficient to degrade generation quality.

4.5 Success and Failure Examples

Success case ($\alpha=0.3$, MLP-only):

Winter snow flutters across the sky,
Snowflakes drift gently down,
Cold wind howls piercingly,
Snowflakes dance in the air.

This output has four lines, each with five characters, rhymed, without repetition.

Failure case 1: Hard superposition $\alpha=1.0$ causing infinite loop. Output begins with winter imagery then enters an endless repetition loop.

Failure case 2: Chained subtraction causing semantic collapse. The model abruptly shifts from poetry to emotional expressions such as "How do you like it? I want to give you more winter poems... I miss you! You are my good friend! I love you!" and then enters a repetition loop.

Failure case 3: Attention-only causing immediate repetition. Output repeats the same four lines over multiple cycles.

5 Discussion

Experimental results show significant differences in injection stability between MLP and Attention layers. We hypothesize that MLP layers primarily store associative knowledge, with weight distributions having high redundancy and robustness; adding bounded deltas does not easily disrupt the overall distribution. In contrast, Attention layers' Q/K/V projections are highly sensitive to numerical magnitude; even small shifts can affect the softmax temperature distribution, triggering attention collapse and manifesting as repetition loops.

Additionally, chained subtraction accumulates floating-point errors across multiple operations; even though mathematically equivalent to single subtraction, it amplifies distributional perturbations in practice. Therefore, we recommend using single direct extraction in practical applications whenever possible.

6 Conclusion

This paper proposes and validates a method for extracting functional plugins from training-stage deltas of homologous models. Experimental results show that: directly extracting the endpoint-startpoint delta (RL-Base) is superior to chained subtraction, which accumulates floating-point errors and causes generation collapse; MLP layers are safe carriers for injection, with deltas that can be stably injected without disrupting generation distribution, while Attention layer deltas tend to trigger repetition loops and are unsuitable for direct injection; the soft superposition protocol ($\alpha=0.3$) is the key to ensuring injection stability, with either too strong or too weak injection leading to failure. This work provides a zero-training, zero-fine-tuning capability reuse solution for modular, pluggable AI systems, with future extensions to larger models and multi-task combination scenarios.

References

- [1] Hu, E. J., et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR, 2022.
- [2] Ilharco, G., et al. Editing Models with Task Arithmetic. ICLR, 2023.
- [3] Wortsman, M., et al. Model Soups: Averaging Weights of Multiple Fine-Tuned Models. ICML, 2022.
- [4] Yadav, P., et al. TIES-Merging: Resolving Interference When Merging Models. NeurIPS, 2023.
- [5] OpenBMB. MiniCPM: A Scalable 1B-Parameter Language Model. Technical Report, 2024.

基于同源模型差值的功能插件提取与软叠加注入

Shizhe Jin

Independent Researcher

摘要：大模型在预训练、指令微调（SFT）和强化学习（RL）等阶段中积累了不同的能力增量，但这些增量通常无法分离复用。本文提出一种简单方法：通过计算同源模型间的参数差值提取“功能插件”，并采用软叠加（缩放系数 α ）注入基座。在 MiniCPM-1B 的诗歌生成任务上，我们发现：① 直接提取 RL-Base 差值优于链式减法；② MLP 层是安全注入载体，Attention 层易引发复读崩溃；③ 最佳方案为 $\alpha=0.3$ 的 MLP-only 注入。该方法为模块化 AI 提供了可行路径。

关键词：参数差值；软叠加；插件注入；指令遵循

1 引言

大规模语言模型通常经历多个训练阶段：预训练（Base）学习广泛的语言知识，指令微调（SFT）赋予其遵循人类指令的能力，强化学习（RL）进一步优化其输出策略。然而，这些阶段所获得的能力增量在模型中相互耦合，难以分离和复用。若能将某一阶段的能力（如指令遵循、格式控制）提取为独立插件，并灵活注入其他基座模型，将极大地提升模型部署的灵活性和效率。

已有研究尝试通过参数差值实现能力迁移（如任务向量），但主要聚焦于不同下游任务间的水平迁移。本文关注同一模型在训练时间线上的纵向演化，并发现直接提取阶段性差值会面临分布偏移问题。为此，我们提出软叠加注入协议，并通过实验验证了其在诗歌生成任务上的有效性。

本文的主要贡献如下：

- （1）提出通过同源模型参数差值提取功能插件的方法，无需额外训练；
- （2）发现直接提取终点-起点差值优于链式减法，后者会累积浮点误差；
- （3）提出软叠加注入协议，并验证了 MLP 层是安全注入载体；
- （4）给出最佳实践参数（MLP-only, $\alpha=0.3$ ），为模块化 AI 提供工程路径。

2 相关工作

参数高效微调方法（如 LoRA、Adapter）通过训练小规模增量矩阵来调整模型行为，但需要额外的计算资源和数据。任务向量（Task Vectors）方法通过计算微调模型与预训练模型的差值来提取任务增量，并可将其加减到其他模型上。模型合并方法（如 TIES-Merging）则尝试将多个微调模型的权重进行融合，以获得更优的泛化性能。

本文与上述方法的区别如下：LoRA/Adapter 的增量来源为训练学出，需要训练，而本文零训练；任务向量的增量来源为跨任务模型相减，虽无需训练但需微调两个模型，

本文关注同模型时间阶段；TIES-Merging 通过对多模型平均实现融合，本文则追求插件化而非融合。本文方法首次聚焦纵向时间演化并引入软叠加协议。

3 方法

3.1 差值提取

给定同一架构的两个模型权重 W_{high} 和 W_{low} （分别代表训练阶段较晚和较早的检查点），我们计算其逐层逐参数的差值：

$$\Delta = W_{high} - W_{low}$$

该差值代表了模型在训练阶段之间获得的能力增量。我们将其称为"功能插件"。

3.2 软叠加注入

将提取到的差值注入到基座模型 W_{base} 时，我们采用软叠加协议：

$$W_{new} = W_{base} + \alpha \times \Delta$$

其中 α 属于 $(0, 1]$ 为缩放系数。我们发现 $\alpha=1$ （硬叠加）会导致分布偏移和生成崩溃，而减小 α 可有效缓解该问题。

3.3 层筛选策略

我们进一步发现，不同网络层对注入的容忍度不同。基于实验验证，我们采用以下策略：仅注入 MLP 层（包括 `gate_proj`, `up_proj`, `down_proj`），不注入 Attention 层（包括 `q_proj`, `k_proj`, `v_proj`, `o_proj`）。MLP 层主要负责存储关联性知识，其权重分布具有较高的冗余度和鲁棒性；而 Attention 层对数值幅度极为敏感，即使微小偏移也可能导致注意力分布崩塌。

4 实验

4.1 实验设置

我们使用 MiniCPM-1B 模型系列，包括 Base（预训练）、SFT（指令微调）和 RL（强化学习）三个版本。任务为五言绝句生成，提示词为"请写一首五言绝句，共四句，每句五个字，押韵，描写冬天"。解码采用束搜索（`num_beams=4`），不使用随机采样。

4.2 基线对比

Base（无插件）的输出为自由诗，散文化，无固定格式，状态为有知识但无指令遵循；原始 SFT 输出完整绝句加注释，作为标准参考；原始 RL 输出完整绝句加注释且更严谨，作为参考基线。

4.3 缩放系数扫描

我们固定使用 MLP-only 插件 (RL-Base 差值, 仅 MLP 层), 测试不同 α 值下的输出质量。当 $\alpha=0.001$ 时, 输出为六言或七言混搭, 出现第二轮对话幻觉, 状态为失败 (注入不足)。当 $\alpha=0.3$ 时, 输出为"冬雪漫天飞, 雪花飘飘落, 寒风凛凛冽, 雪花漫天飞。", 状态为成功。当 $\alpha=0.5$ 时, 输出为五言但"雪花漫天飞"重复出现, 状态为部分成功。当 $\alpha=0.7$ 时, 重复加剧, 状态为失败。当 $\alpha=1.0$ 时, 出现无限循环, 状态为失败。结果表明, $\alpha=0.3$ 是最佳选择, 能够实现稳定、格式正确的生成。

4.4 不同插件类型对比

我们进一步对比了不同提取方式和不同层的插件效果。MLP-only (RL-Base) 采用单次减法, 结果为稳定输出, 最佳; Attention-only (RL-Base) 采用单次减法, 结果为立即复读循环; 完整插件 (RL-Base) 采用单次减法, 结果为重复崩溃; 链式插件 (RL-SFT + SFT-Base) 采用两次减法累加, 结果为语义崩塌, 出现无关内容。关键的发现是: 直接提取 RL-Base 优于链式减法。后者虽然在数学上等价, 但两次浮点运算累积的误差足以破坏生成质量。

4.5 成功与反例输出

成功案例 ($\alpha=0.3$, MLP-only):

冬雪漫天飞,

雪花飘飘落,

寒风凛凛冽,

雪花漫天飞。

该输出为四句, 每句五字, 押韵, 无重复。

反例 1: 硬叠加 $\alpha=1.0$ 导致无限循环。输出为:

雪花飞舞漫天舞,

寒气袭人心阵阵。

冬天来了寒意无边,

万物凋零寂静无声。

雪花飞舞漫天舞,

寒气袭人心阵阵。

(无限循环)

反例 2：链式减法导致语义崩塌。输出中模型从写诗突然转向情感表达，出现“我觉得怎么样？我还想给你更多的冬日的诗歌...我想你了！你是我的好朋友！我爱你！”等无关内容，随后陷入重复循环。

反例 3：Attention-only 导致立即复读。输出为：

雪花飞舞漫天飞，

寒风呼啸凛冽飞，

白雪皑皑银装飞，

寒气袭人寒意飞。

雪花飞舞漫天飞，

寒风呼啸凛冽飞，

(重复循环)

5 讨论

实验结果表明，MLP 层与 Attention 层在注入稳定性上存在显著差异。我们推测原因在于：MLP 层主要负责存储关联性知识，其权重分布具有较高的冗余度和鲁棒性，添加有界差值不易破坏整体分布；而 Attention 层的 Q/K/V 投影对数值幅度极为敏感，微小偏移即可影响 softmax 温度分布，从而触发注意力坍塌，表现为生成重复循环。

此外，链式减法因多次浮点运算累积误差，即使数学上等价于单次减法，工程上仍会放大分布扰动。因此，我们建议在实际应用中尽可能使用单次直接提取。

6 结论

本文提出并验证了一种从同源模型训练阶段差值中提取功能插件的方法。实验结果表明：直接提取终点-起点差值 (RL-Base) 优于链式减法，后者会累积浮点误差导致生成崩溃；MLP 层是安全注入载体，其差值可稳定注入基座而不破坏生成分布，而 Attention 层差值易引发复读循环，不适合直接注入；软叠加协议 ($\alpha=0.3$) 是保证注入稳定性的关键，过强或过弱均会导致失败。本研究为模块化、可插拔的 AI 系统提供了一种零训练、零微调的能力复用方案，未来可扩展至更大模型和多任务组合场景。

参考文献

- [1] Hu, E. J., et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR, 2022.
- [2] Ilharco, G., et al. Editing Models with Task Arithmetic. ICLR, 2023.
- [3] Wortsman, M., et al. Model Soups: Averaging Weights of Multiple Fine-Tuned Models. ICML, 2022.
- [4] Yadav, P., et al. TIES-Merging: Resolving Interference When Merging Models. NeurIPS, 2023.
- [5] OpenBMB. MiniCPM: A Scalable 1B-Parameter Language Model. Technical Report, 2024.